SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I **B Sandhya(192311292)** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

ANSWERS

1.Scenario-Based Assessment: Black-Box Testing of an Online Banking Fund Transfer System

1. Introduction

An online banking system allows customers to transfer money between bank accounts electronically. The system is experiencing frequent failures, particularly when customers transfer amounts near the minimum and maximum permitted transaction limits. These failures may occur due to incorrect boundary validation, improper input handling, transaction processing errors, or insufficient test coverage.

As a test engineer, the objective is to analyze the existing testing strategy, identify its limitations, apply appropriate black-box testing techniques, evaluate random testing, and design an optimized test approach that improves defect detection and transaction reliability.

2. Analysis of the Scenario

The frequent failures at boundary values indicate that the existing test cases may not adequately cover the limits of the permitted transfer amount.

For example, assume that the banking system allows transfers between ₹100 and ₹50,000 per transaction.

The system should accept valid amounts within this range and reject amounts outside it.

Possible problems include:

- Incorrect validation of minimum and maximum transfer amounts.
- Acceptance of amounts below the minimum limit or above the maximum limit.
- Incorrect handling of zero, negative, or non-numeric inputs.
- Errors caused by decimal values and currency precision.
- Incorrect balance deduction or crediting during a transfer.
- Transaction failures when multiple transfers occur simultaneously.
- Insufficient test coverage due to poorly designed test cases.

Critical analysis: The repeated failures suggest that the existing testing strategy may not systematically cover boundary conditions and invalid input classes. Therefore, the test team should adopt systematic black-box testing techniques instead of relying mainly on random inputs.

3. Evaluation of the Current Test Case Design Strategy

The current testing strategy appears insufficient because the system continues to fail at important transaction boundaries.

Limitations of the existing strategy

1. **Insufficient boundary coverage:** Testing only typical transaction amounts may fail to reveal errors at the minimum and maximum limits.
2. **Limited input classification:** The test cases may not distinguish valid inputs from invalid inputs such as negative values, zero, and amounts exceeding the maximum limit.
3. **Dependence on random testing:** Random inputs may not include the exact boundary values where defects are most likely to occur.
4. **Insufficient negative testing:** Invalid transaction amounts may not be tested systematically.
5. **Incomplete transaction validation:** Testing may focus on the transfer amount without checking the sender's balance, recipient's balance, and transaction status.
6. **Lack of regression testing:** Previously fixed defects may reappear if existing test cases are not executed after code changes.

Evaluation

The current strategy should be improved by introducing equivalence partitioning, boundary value analysis, risk-based prioritization, and regression testing. Random testing may still be used as a supplementary technique.

4. Application of Equivalence Partitioning

Equivalence partitioning is a black-box testing technique that divides input data into groups called equivalence classes. Inputs belonging to the same class are expected to behave similarly.

Assume the following transaction rules:

- Minimum transfer amount: ₹100.

- Maximum transfer amount: ₹50,000.
- Transfer amounts must be positive and valid to two decimal places.
- The sender must have sufficient available balance.
- The recipient account must be valid and active.

Equivalence classes for transfer amounts

Test Case	Input Amount	Equivalence Class	Expected Result
EP1	₹50	Below minimum	Reject transfer
EP2	₹500	Valid amount	Accept if other conditions are satisfied
EP3	₹60,000	Above maximum	Reject transfer
EP4	₹0	Zero amount	Reject transfer
EP5	-₹500	Negative amount	Reject transfer
EP6	ABC	Non-numeric input	Reject input
EP7	₹100.50	Valid decimal amount	Accept if other conditions are satisfied
EP8	₹100.567	More than two decimal places	Reject or apply the explicitly defined currency-rounding rule

Reasoning: Equivalence partitioning reduces the number of test cases by selecting representative values from different input classes. Instead of testing every possible amount, the team tests representative valid and invalid inputs.

However, equivalence partitioning alone may not detect defects that occur only at the exact minimum or maximum limits. Therefore, boundary value analysis is also necessary.

5. Application of Boundary Value Analysis

Boundary value analysis focuses on values at the edges of valid input ranges because programming errors frequently occur at these points.

For a transfer range of ₹100 to ₹50,000, the test engineer should test the values immediately below, at, and immediately above each boundary.

Assuming the system accepts whole-rupee amounts, the test cases are:

Test Case	Transfer Amount	Boundary Condition	Expected Result
BVA1	₹99	Just below minimum	Reject
BVA2	₹100	Minimum boundary	Accept
BVA3	₹101	Just above minimum	Accept
BVA4	₹49,999	Just below maximum	Accept
BVA5	₹50,000	Maximum boundary	Accept
BVA6	₹50,001	Just above maximum	Reject

Why is boundary value analysis important?

Suppose the developer mistakenly implements the validation condition as:

amount > 100 && amount < 50000

This condition rejects ₹100 and ₹50,000 even though both amounts should be accepted.

Boundary value analysis can identify this defect by testing the exact minimum and maximum values.

If the system supports paise, the boundary tests should use the smallest permitted currency increment, such as ₹99.99, ₹100.00, and ₹100.01. Similar tests should be performed around the maximum limit.

Justification: Boundary value analysis is especially effective in this scenario because the reported failures occur near transaction limits.

6. Redesigned Test Cases for the Fund Transfer System

The test suite should cover not only transaction amounts but also account balances, account validity, and transaction processing.

Assume the sender has ₹60,000 available, the recipient account is valid, and no additional transaction restrictions apply.

Test Case ID	Test Scenario	Test Input or Condition	Expected Result
TC01	Transfer minimum amount	₹100	Transfer succeeds
TC02	Transfer below minimum	₹99	Transfer rejected
TC03	Transfer maximum amount	₹50,000	Transfer succeeds
TC04	Transfer above maximum	₹50,001	Transfer rejected
TC05	Transfer zero amount	₹0	Transfer rejected
TC06	Transfer negative amount	-₹500	Transfer rejected
TC07	Transfer with sufficient balance	₹5,000 with ₹60,000 available	Transfer succeeds
TC08	Transfer exceeding available balance	₹60,001 with ₹60,000 available	Transfer rejected
TC09	Transfer entire available balance	₹60,000 with ₹60,000 available	Transfer rejected because it exceeds the per-transaction limit
TC10	Transfer to invalid recipient	Invalid account number	Transfer rejected
TC11	Transfer to inactive recipient	Inactive recipient account	Transfer rejected
TC12	Decimal amount	₹500.50	Transfer succeeds if supported
TC13	Excess decimal precision	₹500.567	Rejected or handled according to the defined currency policy

Test Case ID	Test Scenario	Test Input or Condition	Expected Result
TC14	Duplicate transfer request	Same request submitted twice	Only one transfer is processed
TC15	Transaction failure	Failure occurs after debit is attempted	No inconsistent debit or credit; transaction is rolled back or recovered
TC16	Concurrent transfers	Two simultaneous transfers against the same balance	No overdraft or inconsistent account balance
TC17	Successful transfer verification	Valid transfer of ₹5,000	Sender is debited and recipient is credited by the correct amount
TC18	Insufficient balance	Transfer amount exceeds available balance	Transfer rejected without deducting money

Evaluation of the redesigned test suite

The redesigned test cases provide broader coverage by testing valid and invalid amounts, exact boundaries, account conditions, and transaction integrity.

Each test should verify both the response displayed to the user and the actual financial changes recorded in the system. For example, a rejected transaction must not accidentally debit the sender's account.

This approach improves the likelihood of identifying validation defects, incorrect balance calculations, duplicate transactions, and inconsistent transaction states.

7. Evaluation of Random Testing

Random testing generates test inputs randomly rather than selecting all inputs according to predefined categories or boundaries.

For example, the testing team might randomly generate transfer amounts such as ₹450, ₹2,300, ₹12,500, and ₹28,000.

Advantages of random testing

1. It is useful for generating a large number of inputs quickly.
2. It can discover unexpected combinations of values.
3. It helps identify unanticipated input-handling defects.
4. It can support stress testing when combined with large numbers of generated transactions.
5. It may uncover defects that were not considered during manual test design.

Limitations of random testing

1. It may fail to generate exact boundary values such as ₹100 and ₹50,000.
2. It does not guarantee systematic coverage of valid and invalid equivalence classes.
3. Important but rare scenarios may remain untested.
4. Randomly generated inputs may repeatedly test ordinary values while missing high-risk cases.
5. Without controlled data and repeatable random seeds, reproducing a failure can be difficult.

Example

Suppose 1,000 random test cases generate amounts between ₹101 and ₹49,999. All these amounts fall within the valid range.

Although the team has executed 1,000 tests, it has not tested ₹100, ₹50,000, ₹99, or ₹50,001.

Therefore, a large number of random tests does not necessarily mean that important boundary conditions have been covered.

Should random testing be retained or replaced?

Recommendation: Retain random testing as a supplementary technique, but replace its use as the primary testing strategy with systematic black-box testing.

Equivalence partitioning and boundary value analysis should form the foundation of the test suite because they directly address the reported failures.

Random testing should continue to generate additional inputs for unexpected cases, repeated transactions, and combinations of account conditions. Its limitations can be reduced through controlled random seeds, reproducible test data, and automated failure logging.

Thus, random testing should not be completely discarded, but it should not be relied upon alone.

8. Proposed Structured and Optimized Testing Approach

The following testing process is recommended to maximize defect detection while minimizing unnecessary test execution.

Step 1: Analyze requirements

Document the transaction rules, including minimum and maximum limits, available-balance requirements, currency precision, recipient validation, and transaction failure handling.

The expected behavior must be clearly defined before testing begins.

Step 2: Identify equivalence classes

Divide transfer inputs into valid and invalid categories. Select representative test inputs from each category to ensure that major input conditions are covered.

Step 3: Apply boundary value analysis

Test values immediately below, at, and immediately above the minimum and maximum transaction limits.

Include decimal boundaries if the system supports amounts containing paise.

Step 4: Design risk-based test cases

Prioritize scenarios that could result in financial loss or incorrect account balances.

The highest-priority cases should include:

- Incorrect acceptance or rejection of boundary amounts.
- Duplicate debits or credits.
- Transfers exceeding available balances.
- Failed transfers that leave inconsistent account records.
- Concurrent transfers that cause balance inconsistencies.

Step 5: Automate critical test cases

Automate the core equivalence-partitioning, boundary, balance-validation, and transaction-integrity tests.

Automation allows the same test suite to be executed repeatedly after software changes.

Step 6: Perform integration testing

Verify that the fund transfer module interacts correctly with account management, balance calculation, transaction recording, and notification services.

Check that all components agree on whether the transfer succeeded or failed.

Step 7: Conduct negative and failure-recovery testing

Test invalid account details, insufficient balances, interrupted network connections, timeouts, and repeated requests.

Verify that failed transactions do not leave the system in an inconsistent state.

Step 8: Retain random and stress testing

Generate additional random transaction amounts and execute high-volume or concurrent transaction tests in a controlled test environment.

Use reproducible test data and record the inputs, outputs, and failures.

Step 9: Perform regression testing

After fixing defects, rerun the boundary tests and all related transaction tests to confirm that the corrections have not introduced new problems.

Step 10: Evaluate testing effectiveness

Measure the number of defects detected, boundary coverage, equivalence-class coverage, automated test pass rates, and the number of defects escaping into later testing stages.

These measurements help determine whether the revised strategy is improving test effectiveness.

9. Expected Results

After implementing the proposed strategy, the banking system should demonstrate the following outcomes:

- Correct acceptance of transfers at the minimum and maximum permitted amounts.
- Rejection of amounts outside the allowed range.
- Accurate handling of decimal, zero, negative, and non-numeric inputs.
- Correct validation of available account balances.
- Prevention of duplicate transfers and inconsistent account updates.
- Improved identification of boundary-related defects.
- Greater repeatability and efficiency through automated regression testing.
- Improved confidence in the reliability and integrity of fund transfers.

These outcomes must be confirmed through test execution; they should not be assumed merely because the test strategy has been redesigned.

10. Conclusion

The frequent failures in the online banking system indicate that the current testing strategy does not adequately address critical transaction boundaries. Equivalence partitioning helps reduce the number of test cases by grouping similar inputs, while boundary value analysis targets errors near the minimum and maximum permitted amounts.

Random testing is useful for discovering unexpected defects but cannot guarantee that important boundaries and invalid input classes will be covered. Therefore, it should be retained as a supplementary method rather than used as the primary strategy.

A structured approach combining requirement analysis, equivalence partitioning, boundary value analysis, risk-based prioritization, automation, integration testing, random testing, and regression testing will improve defect detection and help ensure that fund transfers are accurate, secure, and reliable.

2.Scenario-Based Assessment: White Box Testing of a Healthcare Application Module

1. Introduction

Healthcare applications manage sensitive patient information, medical records, prescriptions, appointments, and diagnostic results. Software defects in these applications may lead to incorrect medical information, failed transactions, data loss, or delays in patient care.

In this scenario, the healthcare application has undergone testing, but critical defects were discovered after deployment. This indicates that the existing testing approach may not have provided sufficient code coverage or adequately tested critical execution paths.

White box testing is appropriate because it examines the internal structure, logic, conditions, and execution paths of the application. The objective is to identify gaps in the existing test strategy and design a more effective approach that improves reliability, safety, and software quality.

2. Evaluation of the Existing Testing Approach

The occurrence of critical defects after deployment suggests that the testing process was insufficient to identify important software failures before release.

Possible weaknesses in the existing approach

1. **Insufficient code coverage:** Some statements, branches, or execution paths may not have been executed during testing.
2. **Limited conditional testing:** Conditions involving patient validation, authorization, and medical data processing may not have been tested with both true and false outcomes.
3. **Missing negative test cases:** Invalid patient IDs, missing medical information, unauthorized access, and incorrect input values may not have been adequately tested.
4. **Incomplete exception handling:** Database failures, network interruptions, and unexpected input conditions may not have been considered.
5. **Inadequate integration testing:** Interactions between patient records, authentication, databases, and diagnostic modules may have contained undetected defects.
6. **Weak regression testing:** Changes made during development may have introduced new defects that were not identified before deployment.
7. **Poor risk prioritization:** Critical healthcare functions may not have received sufficient testing compared with less important features.

Critical evaluation

The main problem is not necessarily the number of test cases but the adequacy of their design.

For example, a test case may successfully execute the patient validation function without testing what happens when the patient ID is invalid. Similarly, a function may execute every statement while some decision outcomes remain untested.

Therefore, the testing team should adopt a systematic white box testing strategy based on control flow, decision coverage, path analysis, and risk-based test prioritization.

3. Understanding Test Adequacy Criteria

Test adequacy criteria determine whether the executed test cases provide sufficient coverage of the software's internal structure and behavior.

The major criteria relevant to this scenario are statement coverage, branch coverage, and path coverage.

A. Statement Coverage

Statement coverage measures the percentage of executable statements executed by the test suite.

Formula:

Statement Coverage = (Number of statements executed / Total executable statements) × 100

Example:

If a healthcare module contains 20 executable statements and the tests execute 16 of them:

Statement Coverage = (16 / 20) × 100 = 80%

This means that 80% of the executable statements have been executed.

Limitation: Statement coverage does not guarantee that every decision outcome has been tested. A statement may execute under one condition while another important branch remains untested.

B. Branch Coverage

Branch coverage measures whether each possible outcome of a decision has been executed.

For example:

```
if (patientValid)
```

The decision has two outcomes:

- True: The patient is valid.
- False: The patient is invalid.

Both outcomes must be executed to achieve branch coverage for this decision.

Formula:

Branch Coverage = (Number of branches executed / Total branches) × 100

Branch coverage is generally stronger than statement coverage because it examines decision outcomes as well as statement execution.

However, branch coverage does not guarantee that every possible combination of multiple conditions or every execution path has been tested.

C. Path Coverage

Path coverage measures the execution of different paths through the program's control flow graph.

For example, a healthcare application might have these paths:

- Valid patient and successful database response.
- Valid patient and database failure.
- Invalid patient.
- Unauthorized user.
- Missing medical information.

Path testing helps identify defects caused by particular combinations of conditions and execution sequences.

Limitation: Complete path coverage can be impractical because loops may create infinitely many possible paths, while complex programs can have an enormous number of combinations.

Comparison of coverage criteria

Criterion	What It Measures	Strength	Limitation
Statement coverage	Execution of statements	Identifies unexecuted code	May miss decision outcomes
Branch coverage	Execution of decision outcomes	Detects more conditional logic defects	May miss combinations of conditions
Path coverage	Execution of control flow paths	Helps identify path-dependent defects	Complete coverage may be infeasible
Condition coverage	True and false outcomes of individual Boolean conditions	Helps identify condition-related defects	Does not necessarily cover every combination
Modified Condition/Decision Coverage (MC/DC)	Whether each condition independently affects a decision outcome	Useful for high-assurance safety-critical software	Requires more careful test design

Assessment: For a healthcare application, statement coverage alone is inadequate. Branch coverage should be a minimum target for critical decision logic, with selected path coverage and MC/DC applied where the risk and complexity justify them.

4. Construction of a Conceptual Control Flow Graph (CFG)

A Control Flow Graph represents the execution sequence of a program using nodes and directed edges.

- **Nodes:** Represent statements, processing steps, or decision points.
- **Edges:** Represent possible transfers of control between nodes.
- **Decision nodes:** Represent conditions that can produce multiple outcomes.

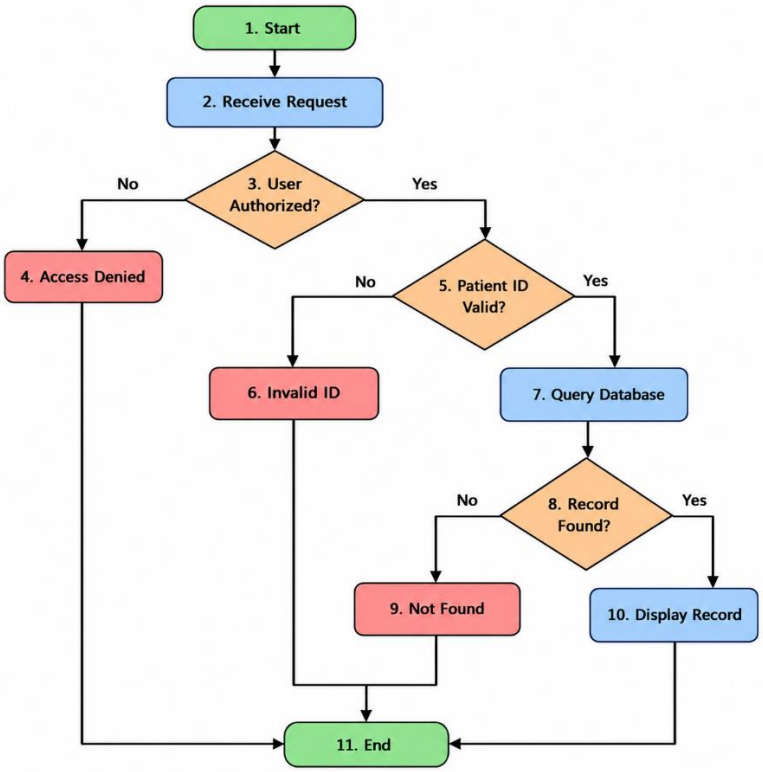
Consider a simplified healthcare module that retrieves a patient's medical record.

Example logic

The module performs the following operations:

1. Receive the patient ID and user credentials.
2. Check whether the user is authorized.
3. Validate the patient ID.
4. Retrieve the patient's record from the database.
5. Check whether the record exists.
6. Display the medical record if successful.
7. Display an appropriate error message if a validation or retrieval step fails.

Conceptual Control Flow Graph



Note: This is a conceptual graph. In a real implementation, database exceptions, timeouts, incomplete records, and other error-handling paths should also be represented.

Explanation of the graph

The graph contains three major decision points:

1. **Authorization decision:** Determines whether the user can access the module.
2. **Patient validation decision:** Determines whether the supplied patient ID is valid.
3. **Record existence decision:** Determines whether the requested medical record exists.

Each decision has true and false outcomes. Testing only the successful execution path would leave several important error-handling paths untested.

5. Identification of Missing Execution Paths

The existing testing approach may have focused mainly on successful requests. The CFG helps identify the additional paths required for adequate testing.

Required execution paths

Path ID	Execution Path	Expected Result	Importance
P1	Start → Request → Unauthorized → Access Denied → End	Access is denied	High
P2	Start → Request → Authorized → Invalid Patient ID → Error → End	Invalid input is rejected	High
P3	Start → Request → Authorized → Valid ID → Record Not Found → Error → End	Appropriate not-found message appears	High
P4	Start → Request → Authorized → Valid ID → Record Found → Display Record → End	Correct record is displayed	High
P5	Authorized → Valid ID → Database failure → Error handling → End	Failure is handled safely	Critical
P6	Authorized → Valid ID → Record found → Incomplete record → Validation or safe error handling	Incomplete information is not incorrectly presented as complete	High
P7	Unauthorized user → Attempt to access another patient's record	Access remains denied	Critical

The first four paths correspond to the main authorization, validation, and retrieval decisions. The remaining paths extend the conceptual graph to cover database exceptions, incomplete data, and patient-data access control.

Analysis of missing paths

1. Unauthorized access path

If testing covers only authorized users, the system may contain a security defect that allows unauthorized users to view medical records.

2. Invalid patient ID path

If invalid IDs are not tested, the application may crash, return incorrect data, or generate an unhandled exception.

3. Record-not-found path

If only existing records are tested, the application may fail to handle missing records correctly.

4. Database failure path

A database timeout or connection failure may cause the application to display misleading information or leave the request in an inconsistent state.

5. Incomplete record path

A record may exist but lack required information. The system must handle this situation without incorrectly presenting the record as complete.

6. Cross-patient access path

The application must verify that users cannot retrieve records belonging to patients outside their authorized scope.

Conclusion: The CFG identifies the paths that need to be covered. It does not, by itself, prove that a defect exists; the test team must execute the corresponding tests and compare the actual results with the expected behavior.

6. Critical Assessment of Achieved Coverage

Suppose the original testing report states that statement coverage reached 90%.

This result alone does not prove that the application is adequately tested.

For example, the application may execute most statements while failing to test the false outcome of the authorization decision or the database failure path.

Consider the following hypothetical coverage report:

Coverage Criterion	Hypothetical Existing Coverage	Assessment
Statement coverage	90%	Some statements remain unexecuted
Branch coverage	60%	Important decision outcomes may be missing
Path coverage	35% of selected feasible paths	Several execution paths remain untested

These percentages are illustrative only; the scenario does not provide actual coverage measurements.

Why statement coverage is insufficient

High statement coverage may still leave critical branches untested. For instance, a test suite might execute the statement that displays a medical record but never test the unauthorized-access branch.

Why branch coverage is important

Branch coverage requires both outcomes of each decision to be exercised. It is therefore more suitable for validating authorization, patient-ID validation, and record-existence decisions.

Why complete path coverage is difficult

A real healthcare module may contain loops, nested conditions, exception handling, and multiple interacting inputs. The number of possible paths can grow rapidly.

Therefore, the team should prioritize feasible paths associated with patient safety, data confidentiality, data integrity, and system availability.

Overall assessment

The critical post-deployment defects indicate that the previous adequacy criteria were either insufficient or not effectively applied to high-risk functionality.

The improved strategy should combine statement coverage, branch coverage, condition coverage, and risk-based path testing instead of relying on a single percentage.

7. Improved Testing Strategy

The following structured approach is recommended to increase defect detection and reduce the risk of failures after deployment.

Step 1: Review requirements and identify risks

Identify the critical functions of the healthcare application, including authentication, authorization, patient validation, record retrieval, medical data updates, and database communication.

Prioritize failures that could expose patient information, corrupt medical records, or disrupt essential services.

Justification: Risk-based prioritization ensures that the most harmful defects receive the greatest testing attention.

Step 2: Perform code and control flow analysis

Review the source code and construct CFGs for critical functions.

Identify decision nodes, exception-handling blocks, loops, early returns, and unreachable code.

Justification: CFG analysis provides a systematic view of the execution structure and reveals paths that may not have been considered during initial test design.

Step 3: Design statement and branch coverage tests

Develop tests that execute every reachable statement and both outcomes of every relevant decision.

For authorization, test both authorized and unauthorized users. For patient validation, test valid and invalid IDs. For record retrieval, test existing and missing records.

Justification: This approach reduces the risk that important statements or decision outcomes remain untested.

Step 4: Apply condition coverage and MC/DC where appropriate

For complex Boolean expressions, test individual conditions and, for high-risk decisions, demonstrate that each condition can independently affect the overall decision outcome.

For example, access might depend on authentication, role permission, and patient-specific authorization.

MC/DC is particularly useful where the consequences of incorrect decisions are severe and the complexity of the logic warrants it.

Justification: This detects defects that simple statement coverage may miss and provides stronger evidence for critical decision logic.

Step 5: Perform risk-based path testing

Select feasible execution paths associated with high-risk operations, including unauthorized access, invalid data, missing records, database failures, and successful retrieval.

Use basis-path testing or other systematic path-selection methods to avoid selecting paths arbitrarily.

Justification: Complete path coverage may be infeasible, but carefully selected paths can provide strong coverage of critical behavior.

Step 6: Test exception handling and boundary conditions

Test empty inputs, malformed patient IDs, maximum permitted input lengths, database timeouts, connection failures, and incomplete records.

Verify that the application fails safely and does not expose sensitive details in error messages.

Justification: Exceptional conditions frequently expose weaknesses that normal execution tests fail to detect.

Step 7: Perform integration and security testing

Test interactions between authentication, authorization, the database, medical record services, and the user interface.

Verify that access controls are enforced on the server and that users cannot access another patient's records by modifying an identifier.

Justification: Some defects occur between components rather than within individual functions.

Step 8: Automate unit tests and regression tests

Automate tests for critical functions and rerun them after every relevant code change.

Include previously discovered defects as permanent regression test cases.

Justification: Automation makes repeated testing more consistent, faster, and less prone to human error.

Step 9: Perform system-level validation

Validate complete workflows using representative test data. Confirm that the displayed medical information matches the intended database record and that errors are handled consistently.

Conduct appropriate performance, reliability, and recovery testing for critical workflows.

Justification: White box testing alone cannot verify every external requirement or integration behavior. System-level testing complements internal code analysis.

Step 10: Establish measurable exit criteria

Before release, require:

- 100% statement coverage of the defined, reachable, in-scope critical code.
- 100% branch coverage for critical decision logic, with justified treatment of infeasible or excluded branches.
- Coverage of all identified high-risk feasible execution paths.
- Appropriate condition or MC/DC coverage for selected safety-critical decisions.
- No unresolved critical defects.
- Successful regression, security, integration, and recovery tests.

These targets are proposed release criteria, not claims about the application's existing coverage. Coverage results must be supported by test reports, and high coverage alone must not be treated as proof that the software is defect-free.

8. How the Improved Approach Enhances Reliability and Reduces Risk

The improved strategy contributes to healthcare application reliability in several ways.

Improvement	Benefit	Risk Reduced
Statement coverage	Identifies unexecuted code	Undetected logic defects
Branch coverage	Tests both outcomes of decisions	Incorrect conditional behavior
Condition coverage and MC/DC	Examines individual conditions in complex decisions	Faulty authorization or validation logic
Risk-based path testing	Targets critical execution sequences	Patient-data errors and unsafe failure handling
Exception testing	Validates behavior during failures	Crashes, inconsistent states, and misleading results
Security testing	Checks access restrictions	Unauthorized disclosure of medical records
Regression testing	Detects previously fixed defects that return	Repeated failures after software updates
Integration testing	Verifies interactions between modules	Data inconsistencies and communication failures
Performance and recovery testing	Evaluates behavior under load and during disruption	Service interruptions and incomplete operations

Overall impact

The combined approach provides stronger evidence that critical functionality behaves correctly under normal, invalid, and exceptional conditions.

It also reduces the probability of undetected defects reaching production, improves confidence in release decisions, and supports the confidentiality, integrity, and availability of healthcare information.

However, testing cannot guarantee that all defects have been eliminated. Continuous monitoring, incident analysis, and regression testing remain necessary after deployment.

9. Conclusion

The discovery of critical defects after deployment indicates that the healthcare application requires a more systematic and risk-focused testing approach. The existing testing strategy may have achieved some statement coverage without adequately testing all decision outcomes or critical execution paths.

White box testing provides a way to examine the program's internal structure through control flow graphs, statement coverage, branch coverage, condition coverage, and selected path testing. The conceptual CFG identifies missing paths involving unauthorized access, invalid patient IDs, missing records, and database failures.

The recommended strategy combines systematic coverage analysis, risk-based path selection, exception handling, security testing, integration testing, automation, and regression testing. Although complete path coverage may be impractical, achieving strong coverage of critical decision logic and all identified high-risk feasible paths significantly improves the reliability and safety of healthcare applications.

Final recommendation: Do not rely on a single coverage percentage. Use multiple coverage criteria, prioritize patient-safety and security risks, and require evidence that critical execution paths have been tested before releasing the application.

3.Scenario-Based Assessment: Black-Box Testing of a Retail E-Commerce System

1. Introduction

A retail e-commerce system allows customers to purchase products by entering or selecting the price, quantity, and discount. These inputs are used to calculate the final payable amount.

In this scenario, testers failed to detect errors that occurred when specific input values were combined. This indicates that the existing black-box testing approach did not adequately cover interactions between multiple inputs.

Black-box testing evaluates software functionality using inputs and expected outputs without examining the internal source code. Appropriate techniques include equivalence partitioning, boundary value analysis, decision table testing, and pairwise testing.

The objective is to redesign the test cases to improve input coverage, identify combination-related defects, and ensure accurate price calculations.

2. Analysis of the Existing Testing Approach

The existing testing strategy appears to have focused on individual input values rather than combinations of price, quantity, and discount.

For example, testers might have verified that the price was accepted, the quantity was valid, and the discount was within the permitted range. However, they may not have checked whether these values produced the correct final amount when used together.

Limitations of the current approach

1. Incomplete input coverage: Some valid and invalid input categories may not have been tested.

2. Missing boundary conditions: Values immediately below, at, and above the permitted limits may have been overlooked.

3. Insufficient combination testing: Errors may occur only when particular prices, quantities, and discounts appear together.

4. Lack of systematic rule testing: The existing tests may not cover all important business rules, such as discount eligibility and quantity restrictions.

5. Weak negative testing: Negative prices, zero quantities, excessive discounts, and non-numeric inputs may not have been tested adequately.

6. Limited interaction testing: Individually valid inputs may produce an invalid or incorrect result when combined.

Example

Assume the system calculates the total payable amount using:

Total = (Price × Quantity) − Discount Amount

Suppose the price is ₹1,000, the quantity is 2, and the discount is 10%.

The expected calculation is:

Subtotal = ₹1,000 × 2 = ₹2,000

Discount = 10% of ₹2,000 = ₹200

Final Amount = ₹2,000 − ₹200 = ₹1,800

If the system incorrectly applies the discount to only one unit, it may calculate the final amount as ₹1,900.

Testing the price, quantity, and discount individually would not necessarily detect this defect. The combination must be tested.

Critical analysis: The primary weakness is inadequate test design for input interactions. The solution is to use complementary black-box testing techniques rather than relying on a small set of ordinary input values.

3. Assumptions and Business Rules

To design the test cases, assume the following rules for the e-commerce system:

- Unit price must be greater than ₹0 and not exceed ₹1,00,000.
- Quantity must be an integer between 1 and 100.
- Discount must be between 0% and 50%, inclusive.
- Price may contain up to two decimal places.
- Discount is applied to the subtotal.
- The final payable amount cannot be negative.
- Invalid inputs must be rejected with appropriate error messages.

These are illustrative requirements for this assessment. In an actual project, the test engineer must use the application's documented business rules.

4. Application of Equivalence Partitioning

Equivalence partitioning divides input values into classes that are expected to behave similarly. Instead of testing every possible value, the tester selects representative values from each class.

A. Price input

Class	Input Example	Expected Result
Invalid: zero price	₹0	Reject
Invalid: negative price	-₹100	Reject
Valid: normal price	₹500	Accept
Valid: decimal price	₹499.99	Accept
Invalid: above maximum	₹1,00,001	Reject
Invalid: non-numeric price	ABC	Reject

B. Quantity input

Class	Input Example	Expected Result
Invalid: zero quantity	0	Reject
Invalid: negative quantity	-2	Reject
Valid: minimum quantity	1	Accept
Valid: ordinary quantity	5	Accept
Valid: maximum quantity	100	Accept
Invalid: above maximum	101	Reject
Invalid: fractional quantity	2.5	Reject
Invalid: non-numeric quantity	ABC	Reject

C. Discount input

Class	Input Example	Expected Result
Valid: no discount	0%	Accept
Valid: ordinary discount	10%	Accept
Valid: maximum discount	50%	Accept
Invalid: negative discount	-5%	Reject
Invalid: above maximum	51%	Reject

Class	Input Example	Expected Result
Invalid: non-numeric discount	ABC	Reject

Evaluation

Equivalence partitioning improves efficiency by reducing redundant test cases while ensuring that major valid and invalid input classes are represented.

However, it may not detect defects that occur only at specific limits or when multiple valid inputs interact. Therefore, boundary value analysis and decision table testing are also necessary.

5. Application of Boundary Value Analysis

Boundary value analysis focuses on values at the edges of an input range because errors often occur around these limits.

For the assumed requirements, the tester should examine the minimum and maximum values and their immediate neighbors.

A. Price boundaries

Test Case	Price	Expected Result
P1	₹0.00	Reject
P2	₹0.01	Accept
P3	₹0.02	Accept
P4	₹99,999.99	Accept
P5	₹1,00,000.00	Accept
P6	₹1,00,000.01	Reject

B. Quantity boundaries

Test Case	Quantity	Expected Result
Q1	0	Reject
Q2	1	Accept
Q3	2	Accept
Q4	99	Accept
Q5	100	Accept
Q6	101	Reject

C. Discount boundaries

Test Case	Discount	Expected Result
D1	-1%	Reject
D2	0%	Accept
D3	1%	Accept

Test Case	Discount	Expected Result
D4	49%	Accept
D5	50%	Accept
D6	51%	Reject

Why is boundary value analysis important?

Suppose the developer uses the condition:

quantity > 1 && quantity < 100

This implementation incorrectly rejects quantities of 1 and 100, even though both should be valid.

Testing the exact boundary values reveals this error.

Similarly, discount validation may incorrectly reject 50% or accept 51% if the comparison operators are implemented incorrectly.

Justification: Boundary value analysis is particularly effective for detecting off-by-one errors, incorrect comparison operators, and validation defects.

6. Application of Decision Table Testing

Decision table testing is appropriate when the expected result depends on combinations of input conditions.

In this scenario, price, quantity, and discount interact to determine whether a purchase is valid and what amount the customer must pay.

Decision table

Assume the system validates all three inputs before calculating the final amount.

Condition or Action	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5
Price is valid	Yes	No	Yes	Yes	Yes
Quantity is valid	Yes	Yes	No	Yes	Yes
Discount is valid	Yes	Yes	Yes	No	Yes
Calculate final amount	Yes	No	No	No	Yes
Display validation error	No	Yes	Yes	Yes	No
Apply discount	Yes	No	No	No	No discount

Rule 5 represents a valid purchase with a 0% discount.

Interpretation

- **Rule 1:** All inputs are valid and a discount is applied.
- **Rule 2:** The price is invalid, so the purchase is rejected.
- **Rule 3:** The quantity is invalid, so the purchase is rejected.

- **Rule 4:** The discount is invalid, so the purchase is rejected.
- **Rule 5:** All inputs are valid, but the discount is 0%; the subtotal is payable without a discount.

The table should be expanded when requirements contain additional rules, such as product-specific discount eligibility, minimum purchase amounts, coupon restrictions, or maximum order values.

Example test case

Input:

- Price = ₹1,000
- Quantity = 3
- Discount = 20%

Calculation:

Subtotal = ₹1,000 × 3 = ₹3,000

Discount = 20% of ₹3,000 = ₹600

Final payable amount = ₹3,000 − ₹600 = ₹2,400

Expected result: The system should display a final payable amount of ₹2,400.

If the system displays ₹2,800 because it applies the discount to only one unit, the test identifies a combination-related defect.

Advantages of decision table testing

1. It systematically represents business rules.
2. It identifies missing combinations of conditions.
3. It verifies that each rule produces the correct result.
4. It helps detect incorrect discount application and validation behavior.
5. It provides a clear basis for deriving test cases.

Justification: Decision table testing is especially useful here because the reported defects occur with particular combinations of inputs rather than only with individual values.

7. Additional Black-Box Testing Techniques

A. Pairwise Testing

Pairwise testing selects test cases so that every pair of values across the selected input parameters appears at least once.

For example, the tester may combine representative values for price, quantity, and discount so that important pairs such as maximum quantity with maximum discount are covered.

This technique is useful when the complete combination space is large.

However, pairwise testing does not guarantee that every three-way interaction or every possible input combination will be tested.

Therefore, it should supplement rather than replace boundary and decision table testing.

B. Error Guessing

Based on common application defects, testers should deliberately examine scenarios such as:

- A high price combined with maximum quantity.
- Maximum discount applied to a high-value order.
- Decimal prices multiplied by large quantities.
- Repeated application of the same discount.
- Incorrect rounding of the final payable amount.
- Invalid input entered alongside otherwise valid values.

These tests can identify practical defects that were not explicitly anticipated during initial test design.

C. State Transition Testing

If the e-commerce system includes a cart and checkout workflow, state transition testing can verify that the system moves correctly between states such as:

Cart Created → Items Added → Discount Applied → Order Confirmed → Payment Completed.

For example, a discount should not be applied twice because a customer refreshes the checkout page.

This technique is useful for testing order workflows, but it does not replace systematic input validation.

8. Revised Test Design Strategy

A revised test strategy should combine the techniques according to their strengths.

Step 1: Analyze requirements

Document the valid ranges, input formats, discount rules, rounding rules, and expected calculations.

Clarify whether discounts are percentage-based or fixed amounts and whether additional eligibility conditions apply.

Step 2: Partition input values

Use equivalence partitioning to identify valid and invalid classes for price, quantity, and discount.

Select representative values from each class.

Step 3: Test input boundaries

Apply boundary value analysis to the minimum and maximum values of all three inputs.

Include exact boundary values and neighboring values, such as quantity 0, 1, 2, 99, 100, and 101.

Step 4: Design combination-based tests

Use decision tables to represent business rules and ensure that each significant condition combination produces the correct result.

Use pairwise testing for broader interaction coverage when the number of possible combinations is large.

Step 5: Verify calculations independently

For every valid combination, calculate the expected subtotal, discount, and final amount independently.

Compare the expected results with the values displayed by the application and the values recorded in the order.

Step 6: Test invalid combinations

Include cases where one or more inputs are invalid simultaneously.

For example, use a negative price with an excessive discount, or a valid price with zero quantity and an invalid discount.

Verify that the system rejects the request safely and provides suitable validation feedback.

Step 7: Automate the test cases

Automate the main equivalence classes, boundary cases, decision table rules, and calculation checks.

Use a data-driven test suite so that many combinations can be executed consistently.

Step 8: Perform regression testing

Whenever a defect is fixed, add the failing case to the regression suite.

Rerun the relevant tests after changes to pricing, discount, quantity validation, or checkout logic.

Step 9: Measure test completeness

Evaluate whether:

- Every identified equivalence class has been tested.
- Every defined boundary has been checked.
- Every significant decision-table rule has a corresponding test.
- Required pairwise combinations have been covered.
- Critical calculation and validation defects have been retested.
- Regression tests pass after the changes.

These criteria provide measurable evidence of test adequacy.

9. Covering All Possible Input Combinations

The requirement to cover all possible input combinations must be interpreted carefully.

If price, quantity, and discount each have many possible values, exhaustive testing can require an extremely large number of tests.

For example, if the tester considers 100,000 possible price values, 100 quantity values, and 51 discount percentages, the number of combinations is:

100,000 × 100 × 51 = 510,000,000 combinations.

This is an illustrative calculation rather than the actual number of possible inputs in the application.

Testing every combination may be impractical, especially when decimal prices and invalid input formats are included.

Recommended approach

- Use equivalence partitioning to reduce redundant input values.
- Use boundary value analysis to cover high-risk limits.
- Use decision table testing to cover defined business rules.
- Use pairwise testing to cover interactions efficiently.
- Use exhaustive testing for small, finite, high-risk input domains where feasible.
- Use automated property-based or randomized testing as supplementary techniques for discovering unexpected combinations.

For example, a useful property-based test would verify that every valid order produces a non-negative payable amount and that the calculated discount never exceeds the permitted discount on the subtotal.

The testing team should distinguish between complete coverage of defined test requirements and exhaustive coverage of every possible input combination. These are not the same.

10. Expected Improvements

Improvement	Expected Benefit
Equivalence partitioning	Better coverage of valid and invalid input categories
Boundary value analysis	Detection of limit-related validation defects
Decision table testing	Detection of errors involving combinations of business rules
Pairwise testing	Efficient coverage of interactions among input parameters
Independent calculation checks	Identification of incorrect subtotal, discount, and final amount calculations
Automation	Faster and more consistent execution
Regression testing	Prevention of previously fixed defects from returning
Coverage tracking	Clearer evidence of test completeness

These improvements increase the likelihood of identifying defects before deployment, although no testing strategy can guarantee the discovery of every possible defect.

11. Conclusion

The failure to detect combination-related defects indicates that the existing black-box testing approach did not provide sufficient coverage of interactions among price, quantity, and discount.

Equivalence partitioning helps identify representative valid and invalid input classes, while boundary value analysis targets defects around input limits. Decision table testing systematically verifies combinations of business conditions, and pairwise testing improves interaction coverage when exhaustive testing is impractical.

The recommended strategy combines these techniques with independent calculation checks, automated testing, and regression testing. Exhaustive testing should be used when the input domain is sufficiently small; otherwise, structured selection and coverage criteria should guide test design.

Final recommendation: The testing team should not simply increase the number of test cases. It should design test cases systematically to cover input classes, boundary conditions, business-rule combinations, and important parameter interactions. This approach improves defect detection, test efficiency, and confidence in the accuracy of the e-commerce system.

4.Scenario-Based Assessment: White Box Path Testing of a Software Module with Nested Conditions and Loops

1. Introduction

White box testing is a software testing technique that examines the internal structure, logic, control flow, and execution paths of a program. It helps identify logical errors, incorrect conditions, loop defects, and unreachable code.

In this scenario, a software module contains complex nested conditions and loops, but the testing team has achieved only partial path coverage. This indicates that some possible execution paths have not been adequately tested.

The main objective is to evaluate the existing testing strategy, identify weaknesses in control flow graph construction and independent path identification, and propose an improved path testing strategy that provides strong coverage of critical execution paths.

2. Evaluation of the Current White Box Testing Approach

The current testing approach is insufficient because partial path coverage means that some execution paths remain untested.

Major limitations

1. Incomplete Control Flow Graph construction: The testing team may have omitted decision nodes, loop-back edges, exception paths, or conditional branches when constructing the CFG.

2. Inadequate independent path identification: The team may not have identified a sufficient set of linearly independent paths through the program.

3. Insufficient testing of nested conditions: Nested if, else if, and else statements may not have been tested with all relevant combinations of conditions.

4. Incomplete loop testing: Loops may have been tested only for normal execution without checking zero iterations, one iteration, multiple iterations, and termination boundaries.

5. Overreliance on statement coverage: Executing most statements does not guarantee that every decision outcome or critical execution path has been tested.

6. Missing exceptional paths: Invalid inputs, unexpected conditions, and error-handling paths may not have been included in the test design.

Critical evaluation

The main weakness is the lack of a systematic relationship between the program's control flow, its independent paths, and the selected test cases.

The team should reconstruct the CFG, calculate cyclomatic complexity, identify independent paths, and develop test cases that exercise the important paths.

However, complete path coverage must be interpreted carefully because loops and nested conditions can produce an enormous or infinite number of possible execution paths.

3. Construction of a Control Flow Graph (CFG)

A Control Flow Graph represents the possible execution flow of a program.

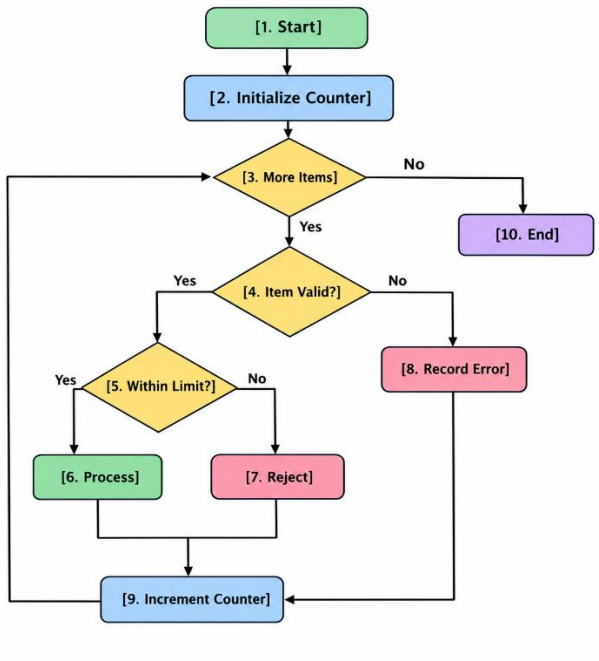
- **Nodes:** Represent statements, processing blocks, or decision points.
- **Edges:** Represent the possible flow of control between nodes.
- **Decision nodes:** Represent conditions that lead to different execution paths.
- **Loop-back edges:** Represent the return from a loop body to its condition.

Consider the following example of a software module that processes a collection of transactions.

Sample program logic

1. Start the program.
2. Initialize the transaction counter.
3. Check whether transactions remain.
4. Validate each transaction.
5. If valid, check whether the transaction amount exceeds a specified limit.
6. Process the transaction if it is within the limit.
7. Otherwise, reject the transaction.
8. If invalid, record a validation error.
9. Increment the counter and repeat the loop.
10. End the program after all transactions are processed.

Conceptual Control Flow Graph



Note: This is a conceptual CFG for explaining path testing. A real CFG should reflect the actual program's statements, conditions, exception handling, and loop structure.

Importance of CFG construction

The CFG provides a visual representation of the program's possible execution routes. It helps the tester identify:

- Missing branches.
- Nested decision outcomes.
- Loop-entry and loop-exit paths.
- Paths that terminate early.
- Error-handling routes.
- Potentially unreachable code.

An incomplete CFG may cause the team to overlook important execution paths, resulting in undetected defects.

4. Identification of Independent Paths

An independent path is a path through the CFG that introduces at least one new edge not included in the previously selected independent paths.

A basis set of independent paths provides a systematic foundation for structural path testing.

For the example above, the following paths represent the major control flow alternatives.

Independent path examples

Path 1: No transactions available

Start → Initialize Counter → More Items? (No) → End.

This tests the loop-exit condition when no items remain.

Path 2: Valid transaction within the limit

Start → Initialize Counter → More Items? (Yes) → Item Valid? (Yes) → Within Limit? (Yes) → Process → Increment Counter → Loop condition.

This tests successful transaction processing.

Path 3: Valid transaction exceeding the limit

Start → Initialize Counter → More Items? (Yes) → Item Valid? (Yes) → Within Limit? (No) → Reject → Increment Counter → Loop condition.

This tests the rejection of transactions that exceed the limit.

Path 4: Invalid transaction

Start → Initialize Counter → More Items? (Yes) → Item Valid? (No) → Record Error → Increment Counter → Loop condition.

This tests validation failure handling.

These paths cover the major decision outcomes represented in the conceptual graph. Because the example contains a loop, each path should also be exercised in appropriate iteration contexts, such as the first iteration and a later iteration.

Why independent path identification matters

If the team does not identify independent paths systematically, it may execute the successful processing path repeatedly while leaving validation failures or limit violations untested.

Independent path analysis provides a structured method for selecting tests that exercise distinct control flow behavior rather than merely increasing the number of test cases.

5. Cyclomatic Complexity and Its Role in Path Testing

Cyclomatic complexity measures the number of linearly independent paths in a program's control flow graph.

It is useful for estimating the minimum number of independent paths required for a basis-path testing approach.

The formula is:

$V(G) = E - N + 2P$

Where:

- **V(G)** = Cyclomatic complexity.
- **E** = Number of edges in the CFG.
- **N** = Number of nodes in the CFG.
- **P** = Number of connected components, normally 1 for a single connected program graph.

For a single connected graph, the formula simplifies to:

V(G) = E − N + 2

An equivalent method for a structured program is:

V(G) = Number of decision points + 1

For the conceptual example, the decision points are:

1. More Items?
2. Item Valid?
3. Within Limit?

Therefore:

V(G) = 3 + 1 = 4.

This indicates that four linearly independent paths are needed for a basis set for the simplified graph.

Important: Cyclomatic complexity does not mean that only four tests are sufficient to test every possible execution path. It estimates the number of independent paths needed for a basis set, not the total number of possible paths, and it does not guarantee defect-free software.

6. Impact of Inadequate Independent Path Identification

Failure to identify independent paths can significantly reduce defect detection.

A. Missed conditional defects

If the invalid-transaction branch is never executed, defects in validation or error handling may remain undetected.

B. Incorrect nested-condition behavior

If only valid transactions within the permitted limit are tested, the team may fail to discover incorrect behavior when the transaction amount exceeds the limit.

C. Loop termination defects

If the loop is tested only with multiple valid items, the team may miss infinite loops, incorrect termination conditions, or errors involving empty input collections.

D. Incorrect counter updates

If the counter is incremented incorrectly on one branch, the loop may skip an item or process the same item repeatedly. Testing each major branch helps expose such defects.

E. Unhandled exceptional behavior

If error-handling paths are absent from the CFG and test suite, unexpected inputs or runtime failures may cause the program to terminate incorrectly.

Overall impact

Inadequate independent path identification reduces confidence in the test results. The software may appear to function correctly under ordinary inputs while still containing defects in less common but important execution paths.

7. Coverage Criteria for Evaluating the Existing Strategy

The team should evaluate coverage using multiple criteria rather than relying only on path coverage.

Coverage Criterion	Purpose	Limitation
Statement coverage	Verifies execution of statements	May miss decision outcomes
Branch coverage	Verifies true and false outcomes of decisions	Does not guarantee every path or condition combination
Condition coverage	Verifies true and false outcomes of individual Boolean conditions	Does not necessarily cover all combinations of conditions
Basis-path coverage	Exercises a basis set of linearly independent paths	Does not cover every possible path
Complete path coverage	Attempts to execute every possible path	Often infeasible with loops and complex logic
Loop coverage	Tests important loop iteration and termination behaviors	Selected iteration counts cannot cover every possible loop execution

Critical assessment

Statement coverage alone is not sufficient for complex nested logic. Branch coverage provides stronger evidence that decision outcomes have been exercised.

Basis-path testing provides a systematic way to select independent paths. Loop testing should complement it by checking important iteration counts and termination behavior.

For high-risk Boolean expressions, condition coverage or Modified Condition/Decision Coverage (MC/DC) may also be appropriate.

Complete path coverage is generally impossible for programs with unbounded loops or an enormous number of paths. The practical objective is to cover all identified critical feasible paths and provide evidence of adequate structural coverage.

8. Improved Path Testing Strategy

The following steps provide a structured approach to improving test adequacy.

Step 1: Analyze the source code

Review the module's nested conditions, loops, exception handlers, early returns, and termination conditions.

Identify all decision points and determine which inputs influence them.

Justification: Understanding the actual control structure reduces the risk of omitting important branches.

Step 2: Construct and validate the CFG

Create a CFG that includes all relevant nodes and edges, including loop-back edges and error-handling paths.

Verify that every decision has its possible outcomes represented.

Justification: An accurate CFG forms the foundation for identifying independent paths and designing structural tests.

Step 3: Calculate cyclomatic complexity

Calculate V(G) using the CFG or the number of decision points plus one for a structured, single-entry program.

Use the result to estimate the number of independent paths needed for basis-path testing.

Justification: Cyclomatic complexity helps quantify control flow complexity and guides systematic path selection.

Step 4: Identify independent paths

Select a basis set of linearly independent paths that collectively exercises the graph's independent control flow structure.

For each path, determine the input conditions needed to execute it.

Justification: This reduces arbitrary test selection and helps ensure that distinct branches are covered.

Step 5: Design test data for nested conditions

Create test cases for each relevant combination of conditions, including:

- All conditions true.
- Each decision producing a false outcome.
- Valid input combined with invalid input.
- Boundary values that change the outcome of a decision.
- Inputs that trigger error-handling branches.

Where complex Boolean conditions exist, add condition coverage or MC/DC tests according to risk.

Justification: These cases help reveal defects that may be hidden when only the normal execution route is tested.

Step 6: Perform systematic loop testing

For each loop, test the following situations where applicable:

- Zero iterations.
- One iteration.
- Two iterations.
- Typical multiple iterations.
- The maximum permitted iteration count.
- Inputs at the loop termination boundary.
- Invalid input or errors during iteration.

For nested loops, prioritize high-risk interactions and representative combinations of inner and outer loop iteration counts.

Justification: These tests help identify loop initialization errors, incorrect termination, counter defects, and repeated-processing problems.

Step 7: Test infeasible and unreachable paths

Determine whether any apparent CFG paths are infeasible because no possible input can satisfy their conditions.

Document the reason for excluding them and investigate unreachable code for potential implementation defects.

Justification: This prevents misleading coverage results and distinguishes genuinely missing tests from impossible execution paths.

Step 8: Automate structural testing

Use unit-testing frameworks and coverage tools to measure statement, branch, and condition coverage where supported.

Integrate these tests into the development pipeline and execute them after relevant code changes.

Justification: Automation improves repeatability, accelerates regression testing, and makes coverage gaps easier to identify.

Step 9: Perform regression testing

Whenever a defect is corrected, preserve the test that exposed it and rerun the relevant existing tests.

Include previously failing nested-condition and loop scenarios in the regression suite.

Justification: This reduces the likelihood that fixes introduce new defects or that earlier defects reappear.

Step 10: Establish release criteria

Before release, require:

- All identified critical feasible paths to be tested.
- Both outcomes of each relevant critical decision to be covered.
- Appropriate loop termination and iteration tests to pass.
- All reachable critical statements to be executed by the test suite.
- Unreachable and infeasible code to be reviewed and documented.
- No unresolved critical defects.
- Successful regression testing.

These criteria should be supported by test results and coverage reports rather than assumptions.

9. Expected Outcomes of the Improved Strategy

Improvement	Expected Outcome
Accurate CFG construction	Better visibility of execution flow
Independent path identification	More systematic structural test selection
Cyclomatic complexity analysis	Better estimation of basis-path testing requirements
Branch and condition coverage	Improved detection of conditional logic defects
Loop testing	Better detection of termination and iteration errors
Risk-based path selection	Greater attention to critical execution scenarios
Automated coverage measurement	More reliable evidence of test adequacy
Regression testing	Reduced risk of previously fixed defects returning

The improved strategy should increase the likelihood of discovering hidden logic defects before deployment and provide stronger evidence that important execution paths behave as expected.

However, coverage measures execution, not correctness. Even 100% statement or branch coverage cannot guarantee that all defects have been detected.

10. Conclusion

The partial path coverage reported in this scenario indicates that the current white box testing strategy requires improvement. Complex nested conditions and loops demand systematic CFG construction, independent path identification, and carefully designed test data.

Cyclomatic complexity helps determine the number of independent paths required for a basis-path testing approach, while statement, branch, condition, and loop coverage provide complementary measures of test adequacy.

The recommended strategy combines CFG analysis, basis-path testing, boundary-focused condition testing, loop testing, automation, risk-based prioritization, and regression testing.

Final recommendation: The testing team should aim to test every identified critical feasible path rather than claim complete coverage of every mathematically possible path. This approach improves defect detection, provides measurable evidence of test completeness, and increases confidence in the reliability of the software module.

5.Scenario-Based Assessment: Hybrid Testing Strategy for a Financial Application

1. Analysis of the Existing Testing Strategy

Although the financial application uses both black box and white box testing, production defects indicate gaps in test case design, input validation, and code coverage. The team may have tested individual functions without adequately testing their interactions and critical execution paths.

2. Evaluation of Test Adequacy and Coverage

The team should evaluate statement coverage, branch coverage, and path coverage. Statement coverage checks executed statements, branch coverage verifies decision outcomes, and path testing examines execution routes. High coverage alone does not guarantee defect-free software.

3. Application of Black Box Testing

- **Equivalence Class Partitioning (ECP):** Divide inputs into valid and invalid classes, such as valid transaction amounts, negative amounts, and amounts exceeding transaction limits.
- **Boundary Value Analysis (BVA):** Test values immediately below, at, and above transaction limits to detect boundary-related errors.

4. Application of White Box Testing

Construct a **Control Flow Graph (CFG)** to identify decisions and execution paths. Apply path testing to cover critical transaction scenarios, including successful transfers, insufficient balances, invalid accounts, and transaction failures. Use branch coverage to verify decision outcomes.

5. Improved Hybrid Strategy and Conclusion

Combine ECP and BVA with CFG-based path testing, automated regression testing, and risk-based test prioritization. This approach improves input coverage, identifies internal logic defects, and verifies critical transaction paths. Therefore, integrating black box and white box testing systematically reduces production risks and improves the reliability and security of the financial application.

6.Scenario-Based Assessment: BVA and ECP in an HR Payroll System

1. Analysis of the Failure

The HR payroll system calculates monthly bonuses based on years of service and performance ratings. Incorrect payouts at exactly 5, 10, and 15 years indicate possible boundary-related defects, such as incorrect comparison operators or improperly defined service-year ranges.

2. Evaluation of the Current Test Design

The existing test cases may have focused on ordinary service durations without testing exact boundary values. Exploratory testing alone may overlook these specific conditions, resulting in incorrect bonus calculations.

3. Application of Boundary Value Analysis (BVA)

Assuming bonus eligibility changes at 5, 10, and 15 years, test values immediately below, at, and above each boundary.

Years of Service	Expected Test Purpose
4.9, 5, 5.1	Test the 5-year boundary
9.9, 10, 10.1	Test the 10-year boundary
14.9, 15, 15.1	Test the 15-year boundary

Expected results should follow the organization's documented bonus policy.

4. Application of Equivalence Class Partitioning (ECP)

Divide service durations into classes such as below 5 years, 5–9 years, 10–14 years, and 15 years or more. Select representative values from each class and test different performance ratings to verify bonus calculations.

5. Improved Strategy and Conclusion

Combine BVA and ECP to test exact boundaries and representative input classes. These techniques are more systematic and repeatable than exploratory testing alone. They improve defect detection, ensure accurate bonus calculations, and reduce financial errors. Exploratory testing should still be retained as a supplementary technique for discovering unexpected defects.

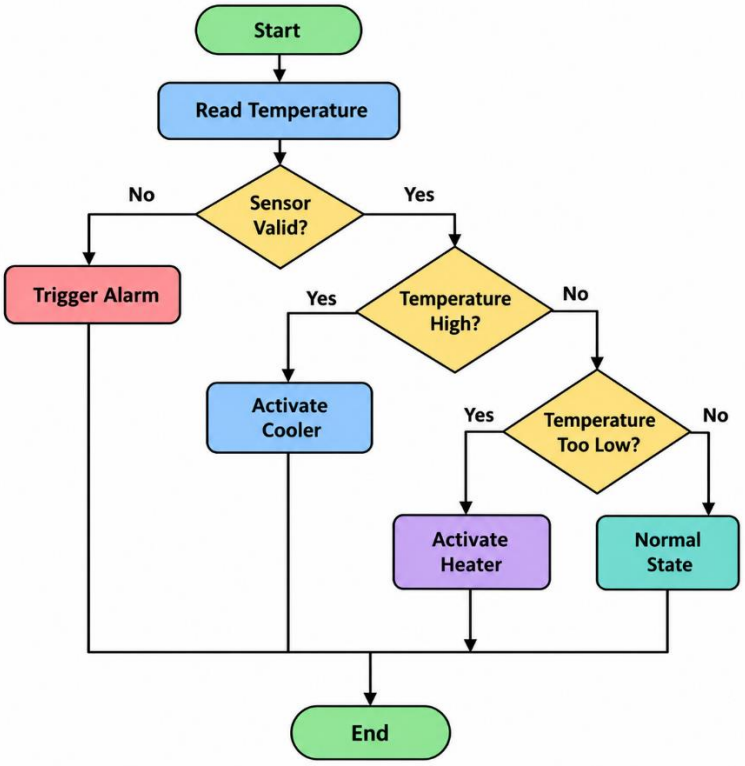
7.Scenario-Based Assessment: White Box Testing and MC/DC in a Laboratory Temperature Control System

1. Analysis of the Failure

The laboratory temperature control system failed to trigger an alarm during a sensor malfunction. This indicates insufficient testing of complex conditional logic and sensor-failure scenarios. The existing testing approach may have tested normal temperature conditions without adequately testing combinations of sensor status and temperature limits.

2. Construction of Control Flow Graph (CFG)

The CFG represents the execution flow of the temperature control module.



3. Evaluation of Branch Coverage

Partial branch coverage means that some decision outcomes were not tested. For example, the team may have tested valid sensor readings but failed to test sensor malfunction combined with extreme temperatures. This can prevent critical alarm defects from being detected.

4. White Box Testing Strategy Using MC/DC

Modified Condition/Decision Coverage (MC/DC) verifies that each individual condition independently affects the overall decision outcome. The testing team should:

- Test valid and faulty sensor readings.
- Test temperatures above, below, and within safe limits.
- Test each condition as true and false.
- Verify that every condition independently changes the relevant decision outcome.
- Test sensor failures during extreme temperature conditions.
- Measure branch coverage and confirm 100% MC/DC for applicable decisions.

For example, if the alarm condition is sensorFault OR temperatureDangerous, test cases must demonstrate that both sensorFault and temperatureDangerous can independently change the alarm decision.

5. Justification and Conclusion

Achieving 100% MC/DC for the applicable decision logic provides stronger evidence that each condition has been tested independently. Combined with CFG analysis, branch testing, and fault-injection tests, this approach helps identify missed alarm conditions and incorrect heater or cooler activation.

Conclusion: Systematic white box testing with CFG analysis and 100% MC/DC improves defect detection and strengthens safety-critical reliability. However, it should be combined with system-level testing and hardware sensor-failure simulations because code coverage alone cannot guarantee complete safety.

8.Decision Table Testing For Promo Code Combinations in a Travel Booking Portal

1. Analysis of Testing Inputs in Isolation

Testing price, promo type, and user tier separately may fail to identify defects caused by their interactions. For example, two promo codes may work correctly individually but produce a negative total when applied together to a premium user's discounted booking. Isolated testing may miss duplicate discounts, incorrect discount calculations, and missing validation rules. Therefore, Decision Table Testing is useful for systematically testing combinations of conditions and their corresponding outcomes.

2. Conditions and Assumptions

Assume the travel booking portal uses the following rules:

- **Booking Price:** Low (₹1,000) or High (₹10,000).
- **Promo Type:** Percentage discount or fixed discount.
- **User Tier:** Regular or Premium.
- Percentage promo: 20% discount.
- Fixed promo: ₹2,000 discount.
- Premium users receive an additional 10% discount.
- Only one promo code is allowed per booking.
- The total payable amount must never be negative.

These values are illustrative; actual tests should follow the portal's documented business rules.

3. Decision Table

Rule	Price	Promo Type	User Tier	Expected Outcome
R1	Low	Percentage	Regular	Valid; total = ₹800
R2	Low	Percentage	Premium	Valid; total = ₹720
R3	Low	Fixed	Regular	Reject or cap discount; total must not be negative
R4	Low	Fixed	Premium	Reject or cap discount; total must not be negative
R5	High	Percentage	Regular	Valid; total = ₹8,000
R6	High	Percentage	Premium	Valid; total = ₹7,200
R7	High	Fixed	Regular	Valid; total = ₹8,000
R8	High	Fixed	Premium	Valid; total = ₹7,200

Note: The fixed-discount premium total assumes the additional 10% discount is calculated after the fixed discount. The business rules must define the discount order explicitly.

4. Optimized Test Set

The eight tests above cover all combinations of the three binary conditions: price category, promo type, and user tier. Each test checks the calculated total and verifies that invalid combinations are handled safely.

Additional targeted tests should include:

1. Apply two promo codes together and verify that the second code is rejected if stacking is prohibited.
2. Apply a fixed discount greater than the booking price and verify that the total never becomes negative.
3. Test a booking price of ₹0 and a price just below, at, and above the minimum eligible price.
4. Verify that premium discounts are applied only to eligible users.
5. Check that repeated promo submissions do not apply the same discount twice.

The test set can be optimized further using risk-based prioritization and pairwise testing, but the full eight-case table should be retained when exhaustive coverage of these three binary factors is required.

5. Conclusion

Decision Table Testing identifies defects caused by interactions among booking price, promo type, and user tier. Combining it with boundary-value testing and promo-stacking validation improves defect detection while reducing redundant tests. The key requirement is that the portal must reject unauthorized promo combinations and ensure that the final payable amount is never negative.

9.Path Testing and Ciclomatic Complexity in a Legacy Database Migration Tool

1. Evaluation of the Current White-Box Approach

The team achieved high statement coverage, meaning most executable statements were executed at least once. However, statement coverage does not guarantee that all decision outcomes, execution paths, or loop termination conditions have been tested. Consequently, nested loops may enter infinite iterations when records are malformed, termination conditions are incorrect, or loop variables are not updated properly.

2. Why Statement Coverage Was Insufficient

- It checks whether statements execute, not whether all possible paths are tested.
- It may miss incorrect loop exit conditions.
- It does not guarantee that nested loops terminate correctly.
- It may overlook empty datasets, single-record datasets, and unusually large datasets.
- It does not ensure that exceptional conditions, such as corrupted records, are handled safely.

3. Proposed Path Testing Strategy

Construct a Control Flow Graph (CFG) representing the program's statements, decisions, and loops. Identify independent execution paths and design test cases to execute each path.

Test the following scenarios:

1. **Empty input:** Verify that all loops terminate without processing records.
2. **Single record:** Check that each loop executes the expected number of times.
3. **Multiple records:** Validate normal processing through all nested loops.
4. **Boundary conditions:** Test zero, one, maximum permitted, and maximum-plus-one records.
5. **Invalid records:** Verify that malformed records do not cause infinite loops.
6. **Loop termination:** Check that loop variables update correctly and that exit conditions eventually become true.
7. **Nested-loop boundaries:** Test inner loops with zero, one, and multiple iterations while also testing outer-loop boundaries.

Use execution timeouts and iteration limits as safeguards during testing. These safeguards help detect hangs but do not replace correcting the underlying loop logic.

4. Cyclomatic Complexity Calculation

Cyclomatic Complexity measures the number of linearly independent paths through a program's control flow.

Formula:

$V(G) = E - N + 2P$

Where:

- **E** = Number of edges in the control flow graph.
- **N** = Number of nodes.
- **P** = Number of connected components, usually 1 for a single program graph.

For a connected graph, it can also be calculated as:

V(G) = Number of decision points + 1

For example, if a migration function contains four decision points, its Cyclomatic Complexity is:

V(G) = 4 + 1 = **5**

This indicates that at least five linearly independent paths are needed for a basis-path testing strategy. The actual number for the migration tool must be calculated from its control flow graph.

5. Justification and Conclusion

Path Testing provides stronger control-flow coverage than statement coverage by testing independent paths and important loop boundaries. Cyclomatic Complexity helps identify the minimum number of independent paths needed for basis-path testing and highlights functions that may require additional testing. Combining path testing, boundary-value testing, malformed-record tests, and execution-time monitoring helps identify infinite-loop defects and improves migration reliability.

10.State Transition Testing for a Mobile Food Delivery App

1. Evaluation of the Current Black-Box Testing Strategy

The app uses black-box testing to verify its functionality without examining the internal code. Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) may validate inputs such as order details, delivery charges, and estimated delivery times. However, these techniques may not detect incorrect transitions between order states. For example, if a driver cancels an order during delivery, the tracker may remain stuck at "Out for Delivery" because the cancellation transition was not tested.

2. Application of State Transition Testing

State Transition Testing verifies how the application moves from one state to another in response to events.

States:

- S1: Ordered
- S2: Preparing
- S3: Out for Delivery
- S4: Delivered
- S5: Cancelled
- S6: Reassignment Pending

3. Redesigned State Transition Test Scenarios

Test Case	Current State	Event	Expected Next State
TC1	Ordered	Restaurant accepts order	Preparing
TC2	Preparing	Food is ready and picked up	Out for Delivery
TC3	Out for Delivery	Delivery completed	Delivered
TC4	Out for Delivery	Driver cancels	Reassignment Pending
TC5	Reassignment Pending	New driver assigned	Out for Delivery
TC6	Reassignment Pending	No driver available	Cancellation or support escalation
TC7	Delivered	Driver cancellation event received late	Remain Delivered
TC8	Cancelled	Delayed delivery update received	Remain Cancelled
TC9	Ordered	Invalid delivery-completed event received	Reject event; remain Ordered
TC10	Out for Delivery	Duplicate driver-cancellation event received	Handle safely without duplicate reassignment

Assumption: The application supports driver reassignment. If reassignment is not possible, the documented business rules should define the appropriate cancellation and refund process.

4. Why ECP and BVA Are Insufficient

Equivalence Class Partitioning (ECP) groups similar inputs into classes, while **Boundary Value Analysis (BVA)** checks values near input limits. These methods are useful for validating input data but do not directly verify the sequence of states and events.

For example, both techniques may confirm that a valid order ID is accepted, but they may not detect that a delayed driver-cancellation event incorrectly changes a delivered order back to an active state. State Transition Testing specifically checks valid transitions, invalid transitions, event ordering, and repeated events.

5. Conclusion

State Transition Testing is essential for dynamic workflows such as food delivery tracking. By testing normal transitions, driver cancellations, reassignment, duplicate events, and delayed updates, it identifies logic gaps that input-focused testing may miss. Combining State Transition Testing with ECP, BVA, and integration testing improves reliability and ensures that the tracker displays the correct order status.

11.Hybrid Testing Strategy for a Multi Factor Authentication Module

1. Critical Assessment of Existing Testing Methods

The authentication module uses password strength, OTP timing, and IP location to verify users. Black-box testing checks authentication behavior using different inputs, while white-box testing examines internal logic, branches, and execution paths. However, brute-force vulnerabilities may remain if testing does not adequately cover repeated login attempts, OTP retries, rate limiting, and interactions between authentication factors. Passing functional tests does not guarantee resistance to automated attacks.

2. Error Guessing Strategy

Error Guessing uses tester experience to anticipate likely defects and security weaknesses. Test the following scenarios in an authorized test environment:

- Repeated incorrect password attempts to verify account throttling or temporary lockout.
- Repeated OTP submissions to check retry limits and OTP expiration.
- Rapid login requests to verify rate limiting and abuse detection.
- Correct passwords combined with expired or invalid OTPs.
- Authentication attempts from unfamiliar IP addresses and suspicious location changes.
- Repeated OTP requests to check whether old codes remain valid.
- Concurrent login attempts to identify race conditions that bypass attempt limits.

3. Formal Path Analysis

Construct a Control Flow Graph (CFG) for the authentication process and identify independent execution paths through its decisions.

Example authentication logic:

1. Validate password.
2. Check password-attempt limit.
3. Validate OTP and expiration time.
4. Check OTP-attempt limit.
5. Evaluate IP location and risk.
6. Grant access, deny access, or require additional verification.

Test paths should include:

- Correct password, valid OTP, and trusted IP: access granted.
- Incorrect password: access denied and attempt counter updated.
- Correct password but invalid or expired OTP: access denied.
- OTP retry limit exceeded: further attempts restricted.
- Suspicious IP: additional verification or access restriction.
- Multiple failed attempts followed by a correct password: access remains subject to throttling and risk policies.
- Simultaneous authentication requests: attempt limits remain effective.

Cyclomatic Complexity can help estimate the number of independent paths that require testing. However, security testing must also examine important combinations of conditions and repeated requests that cannot be fully represented by basic path coverage alone.

4. Proposed Hybrid Testing Strategy

Integrate black-box and white-box methods as follows:

1. **Black-box testing:** Validate authentication outcomes, rate limits, OTP expiration, and error messages without relying on implementation details.
2. **White-box testing:** Examine branches, loops, attempt counters, OTP-validation logic, and exception-handling paths.
3. **Error Guessing:** Target likely weaknesses, including unlimited retries, OTP reuse, inconsistent lockout behavior, and race conditions.
4. **Path Analysis:** Test independent paths and important combinations of password, OTP, and IP-location conditions.
5. **Regression testing:** Repeat security tests after fixes to confirm that the vulnerabilities have not returned.

5. Justification and Conclusion

Black-box testing verifies externally observable security behavior, while white-box testing exposes weaknesses in internal control flow. Error Guessing targets realistic failure scenarios, and Path Analysis provides systematic coverage of important execution paths. Combining these techniques improves the detection of brute-force vulnerabilities, OTP-validation defects, and authentication bypasses.

The module should also enforce server-side rate limiting, progressive delays or temporary lockouts, OTP expiration, secure attempt counters, and monitoring of suspicious activity**.** No single testing technique guarantees security; hybrid testing combined with secure design and continuous monitoring provides stronger protection.

12.Equivalence Class Partitioning For an E-Learning Subscription System

1. Evaluation of Current Testing Criteria

The errors occurring when users upgrade exactly on the expiration date indicate that the current testing criteria may not adequately cover date boundaries and subscription transitions. Testing only typical dates and subscription types may overlook conditions such as upgrading on the last valid day, upgrading after expiration, or changing course access during an upgrade. Therefore, systematic testing of valid and invalid date ranges and subscription combinations is necessary.

2. Equivalence Class Partitioning (ECP)

ECP divides input data into groups expected to behave similarly. The following classes can be used, assuming the subscription expiration date is inclusive.

A. Date Range Classes

Class	Input Condition	Expected Result
D1	Upgrade before expiration date	Upgrade succeeds if otherwise eligible
D2	Upgrade exactly on expiration date	Upgrade succeeds according to the defined policy
D3	Upgrade after expiration date	Expired-subscription rules are applied
D4	Invalid date format	System rejects the input
D5	Missing subscription date	System displays a validation error
D6	Start date later than expiration date	System rejects the invalid date range

B. Subscription Type Classes

Class	Subscription Type	Expected Result
S1	Free	Free-tier access is provided
S2	Basic	Basic-tier access is provided
S3	Premium	Premium-tier access is provided
S4	Invalid subscription type	System rejects the request

3. Structured Test Design

Test Case	Date Condition	Current Tier	Requested Tier	Expected Result
TC1	Before expiration	Free	Basic	Upgrade succeeds
TC2	On expiration date	Basic	Premium	Upgrade succeeds according to the inclusive-date policy
TC3	After expiration	Basic	Premium	Expired-subscription rules are applied
TC4	Before expiration	Basic	Free	Downgrade follows the defined policy
TC5	On expiration date	Free	Premium	Upgrade succeeds if payment and eligibility checks pass
TC6	Invalid date format	Basic	Premium	Validation error is displayed
TC7	Missing expiration date	Premium	Basic	Request is rejected or handled according to the defined policy
TC8	Before expiration	Premium	Premium	Duplicate upgrade is prevented or handled safely
TC9	On expiration date	Basic	Basic	Subscription remains consistent without unintended changes
TC10	After expiration	Free	Basic	New subscription or renewal follows the defined rules

4. Justification of Completeness

ECP reduces redundant tests by selecting representative values from each valid and invalid class. Testing the expiration date explicitly is essential because errors often occur at this boundary. The test design should also cover all relevant combinations of current tier, requested tier, date class, payment status, and course eligibility. Boundary Value Analysis can supplement ECP by testing the day before expiration, the expiration day, and the day after expiration.

5. Conclusion

Combining ECP with boundary-focused testing provides systematic coverage of subscription dates and tier changes. It helps detect incorrect expiration handling, unauthorized course access, and failed upgrades. Clearly defined date rules, consistent subscription updates, and regression testing are necessary to ensure seamless transitions between Free, Basic, and Premium tiers.

13.White-Box Testing Strategy for a Smart-Home Lighting Controller

1. Risks of Relying Solely on Statement Coverage

Statement Coverage measures whether each executable statement runs at least once. However, it does not guarantee that every decision outcome or Boolean condition has been tested. In a smart-home lighting controller, switch-case statements and complex Boolean expressions may contain untested logical paths. These gaps can cause incorrect dimming, unexpected light activation, or failures when sensor readings interact with hardware conditions.

2. Branch Coverage Technique

Branch Coverage verifies that each decision outcome, such as True and False, executes at least once. It also requires testing relevant switch-case alternatives and the default case.

Example: Consider the condition:

```
ambientLight < 100 && motionDetected == true
```

Test cases:

Test	Ambient Light	Motion Detected	Expected Result
TC1	50	True	Lights turn ON
TC2	150	True	Lights remain OFF
TC3	50	False	Lights remain OFF
TC4	150	False	Lights remain OFF

These tests exercise the True and False outcomes and help identify missing decision branches.

3. Condition Coverage Technique

Condition Coverage verifies that every individual Boolean condition evaluates to both True and False at least once.

For the expression:

```
ambientLight < 100 && motionDetected == true
```

The two conditions are:

- C1: Ambient light is below 100.
- C2: Motion is detected.

Tests TC1, TC2, and TC3 ensure that both conditions take True and False values. However, Condition Coverage alone does not guarantee that every decision outcome is exercised; therefore, it should be combined with Branch Coverage.

4. Improved White-Box Testing Strategy

1. **Control Flow Analysis:** Construct a Control Flow Graph (CFG) to identify decisions, switch-case alternatives, and execution paths.
2. **Branch Coverage:** Test every decision outcome, including all relevant switch cases and the default case.
3. **Condition Coverage:** Ensure every individual Boolean condition evaluates to True and False.
4. **Boundary Testing:** Test sensor readings just below, exactly at, and just above dimming thresholds.
5. **Fault Injection:** Simulate disconnected sensors, invalid readings, delayed responses, and hardware communication failures.
6. **Integration Testing:** Verify that software commands correctly control the physical lights and that sensor readings produce the expected brightness.
7. **Regression Testing:** Repeat the test suite after code or hardware-interface changes.

5. Justification and Conclusion

Statement Coverage alone may miss untested branches, Boolean combinations, and hardware error-handling paths. Combining Branch Coverage and Condition Coverage provides stronger evidence that decision logic behaves correctly. Boundary testing and hardware-in-the-loop testing further identify failures caused by sensor inaccuracies, threshold errors, or incorrect control signals. This integrated white-box strategy improves lighting accuracy, reliability, and hardware-software compatibility.

14.Robust Boundary Value Analysis for a Logistics Shipping Cost System

1. Analysis of Current Input Validation Failure

The logistics system calculates shipping costs based on package weight, destination zone, and delivery speed. The failure to detect zero and negative weights indicates that the existing tests focused mainly on valid inputs. Insufficient boundary testing allowed invalid values to bypass validation, potentially causing incorrect shipping charges, calculation errors, or system failures.

2. Boundary Value Analysis (BVA) — Robustness Testing

Boundary Value Analysis tests values around the minimum and maximum limits. Robust BVA also includes invalid values outside those limits.

Assumption: The acceptable package weight is greater than 0 kg and less than or equal to 100 kg.

Test Case	Weight (kg)	Input Category	Expected Result
TC1	-1	Invalid: below minimum	Reject input
TC2	0	Invalid: zero weight	Reject input
TC3	0.01	Valid: minimum positive weight	Calculate shipping cost
TC4	0.02	Valid: just above minimum test value	Calculate shipping cost
TC5	50	Valid: normal value	Calculate shipping cost
TC6	99.99	Valid: near maximum	Calculate shipping cost
TC7	100	Valid: maximum	Calculate shipping cost
TC8	100.01	Invalid: above maximum	Reject input

Note: The actual minimum and maximum weights must follow the system's documented business rules. If the system accepts fractional weights, appropriate precision limits should also be tested.

3. Testing Other Input Parameters

Robust testing should also validate destination zones and delivery speeds.

- **Destination zone:** Test valid zones, unknown zone codes, blank values, and invalid formats.
- **Delivery speed:** Test supported options such as Standard, Express, and Same-Day, along with unsupported values.
- **Combined invalid inputs:** Test negative weight with an invalid destination and zero weight with an unsupported delivery speed.
- **Missing inputs:** Verify that missing weight, destination, or delivery speed produces a clear validation error.

4. Importance of Negative Testing

Negative testing verifies how a system responds to invalid, unexpected, or unacceptable inputs. It helps identify missing validation rules, prevents incorrect shipping charges, and reduces the risk of calculation failures. It also ensures that invalid data does not enter the database or affect downstream logistics operations.

5. Conclusion

Robust Boundary Value Analysis improves input validation by testing values at, below, and above acceptable limits. Combining it with negative testing and validation of destination and delivery options helps prevent invalid shipping calculations. Therefore, testing negative scenarios is essential for maintaining accuracy, reliability, and robustness in logistics management systems.

15.Basis Path Testing For Parallel Medical Imaging Software

1. Evaluation of Random Testing

Random Testing generates test inputs without systematically targeting specific execution paths. Although it can reveal unexpected defects, it may fail to identify problems associated with prime-number dataset sizes because those inputs might not be selected frequently. In parallel processing, prime-sized datasets may expose incorrect loop bounds, uneven workload distribution, indexing errors, or synchronization problems. Therefore, relying only on random testing is insufficient for this specialized mathematical module.

2. Proposed White-Box Testing Approach

White-box testing examines the internal program structure, including loops, conditions, and execution paths. The following approach is recommended:

1. **Control Flow Graph (CFG):** Construct a graph showing program statements, decisions, parallel loops, and exit conditions.
2. **Basis Path Testing:** Identify linearly independent execution paths through the CFG and design test cases to execute each path.
3. **Loop Testing:** Test loops with zero, one, two, typical, and maximum permitted iterations wherever applicable.
4. **Dataset Testing:** Test sizes such as 1, 2, 3, 5, 7, 8, 10, and larger representative values to compare prime and non-prime sizes.
5. **Parallel Execution Testing:** Repeat tests with different thread counts and execution schedules to identify race conditions and synchronization defects.

3. Application of Basis Path Testing

Basis Path Testing uses Cyclomatic Complexity to estimate the number of linearly independent paths.

Formula:

$V(G) = E - N + 2P$

For a single connected control flow graph:

V(G) = Number of decision points + 1

For example, if the processing function contains three decision points, its Cyclomatic Complexity is:

V(G) = 3 + 1 = **4**

The team should design at least four independent basis-path tests for that function, then add targeted tests for dataset sizes, parallel execution, and mathematical correctness. The actual number of paths must be calculated from the module's CFG.

4. Ensuring Mathematical Accuracy and System Stability

- Compare parallel-processing results against a trusted sequential implementation or reference dataset.
- Verify output dimensions, pixel values, numerical tolerances, and relevant mathematical invariants.
- Test prime and non-prime dataset sizes, especially sizes that do not divide evenly among worker threads.
- Run repeated executions to detect nondeterministic results.
- Use thread-safety checks and memory diagnostics to identify race conditions, out-of-bounds access, and resource leaks.
- Retain random testing as a supplementary technique for discovering unexpected failures.

5. Conclusion

Random Testing alone may miss defects linked to specific mathematical and control-flow conditions. Combining Basis Path Testing, loop testing, prime-sized dataset tests, and parallel execution checks provides more systematic coverage. Comparing results against trusted mathematical references helps establish numerical accuracy, while repeated parallel tests improve system stability and reliability in medical imaging applications.